

面向 SPARK 的携带证明验证条件：一份经验报告

Tony Gair（盖尔·托尼）

2026 年 8 月 9 日

面向 SPARK 的携带证明验证条件：一份经验报告

Tony Gair · 2026 年 8 月 9 日 预印本。The Dark Factory Ltd, 英国南希尔兹 (South Shields)。

作者: Tony Gair — ORCID 0009-0006-6279-3677。许可协议: CC BY 4.0。

译本说明: 本文为英文原文的简体中文译本; 一切以英文原文为准。工具名、代码标识符、数值、DOI 与参考文献条目保留原文形式。

摘要

SPARK/Ada 的工业部署依赖 `gnatprove` 及其 SMT 后端来消解验证条件 (verification conditions, VC)。然而, 这一裁决本身只是验证器给出的一项断言: 下游若要使用某个已验证的组件, 要么信任生产方的工具链, 要么自行重新跑一遍证明——而后者代价高昂。我们报告一项实验: 为 SPARK 验证中的 SMT 环节附加可独立校验的证明凭证 (**proof certificates**)。我们使用 SPARK 工具链内已随附的 `cvc5` 二进制程序, 产出协作证明演算 (Cooperating Proof Calculus, CPC) 凭证, 并用 **Ethos** 加以校验; **Ethos** 是一个与产生证明的求解器不共享任何代码的独立校验器。在两个工业级 SPARK 组件上, 我们做到了 31 个验证条件中有 28 个被独立校验通过, 其中在一个整数算术税务组件上做到了 13 个中的 13 个, 每份凭证的平均校验开销约为 0.03 秒——比重新证明低三个数量级。那三个失败可归结为单独一处、定位精确的重写。我们报告方法、测量结果、令该流水线脆弱的版本兼容性风险, 以及所得成果究竟能够确立什么、又到何处为止。

1. 引言

一个经过形式化验证的软件组件, 通常连同一项断言一起发布: 生产方声明证明义务已被消解。若接收方希望依赖这一断言, 则有两个选项, 且两者都不理想。其一, 信任生产方、他们的工具链以及求解器配置——这是一种社会性保证, 而非数学性保证。其二, 重新跑一遍验证——这需要同样的工具链、同样的版本, 且每个组件耗费数分钟到数小时的算力。

这正是携带证明代码 (Proof-Carrying Code) 在近三十年前所指出的缺口 [1, 2]: 应由生产方付出这份努力, 而消费方只需廉价地校验一份凭证。PCC 并未成为常态, 通常的解释在供给侧: 为每一件制品产出证明, 都需要一位人类专家。

如今有两件事发生了变化。工业代码的自动化验证在某些领域已成常规; SMT 求解器也获得了细粒度、可机器校验的证明输出。我们提出一个狭窄的经验性问题:

一个由标准工具链验证过的 **SPARK** 组件，能否随附一份第三方可以廉价校验、且无须信任生产方的凭证？

我们的回答是“部分可以”，并对其加以量化，同时精确指出这条链条中仍未被认证的是哪一环。

2. 背景

SPARK 与 **gnatprove**。**SPARK** 是 Ada 的一个可验证子集。**gnatprove** 将带标注的源代码翻译为 Why3 中间语言，通过最弱前置条件（weakest-precondition）演算生成 VC，并交由 SMT 求解器消解（在本文所研究的配置中为 **cvc5**）。

CPC 与 **Ethos**。协作证明演算是一套覆盖 **cvc5** 在其主流理论中所用推理的证明系统，以 **Eunoia** 逻辑框架表达。**Ethos** 是一个针对 **Eunoia** 证明的独立校验器。**Ethos** 并不内置某个固定的演算；演算是以声明式方式提供的，这使得校验器相对于求解器而言体量很小。

可信基（**trust base**）。为 SMT 环节附上一份凭证，可将对求解器的依赖——历史上最庞大、最难以审计的组件——降低为对一个校验器加上一份演算定义的依赖。

3. 方法

对每个组件：

1. 运行 `gnatprove -P <project>.gpr --level=2 --debug`，它会将生成的 VC 以 SMT-LIB 文件形式保留在 `obj/gnatprove/` 目录下。
2. 对每个 VC，运行 `cvc5`，参数为 `--dump-proofs --proof-format-mode=cpc --proof-granularity=dsl-rewrite`。
3. 去掉开头的 `unsat` 行，以及在反驳（refutation）之后才产生的任何末尾（`error ...`）（见 §6.2）。
4. 用 **Ethos** 针对与求解器版本相匹配的 **CPC** 签名集（signature set）加以校验。

证明粒度是决定性的，而且我们认为其文档记载不足。在默认粒度下，凭证并非无漏洞（hole-free）；同样的义务在 `dsl-rewrite` 粒度下则产出完整的凭证：

<code>--proof-granularity</code>	未被论证的步骤（玩具级 QF_LIA 实例）
<code>default</code>	17
<code>macro</code>	11
<code>theory-rewrite</code>	17
<code>dsl-rewrite</code>	0

4. 研究对象

取自一个工业级已证明核心语料库中的两个 **SPARK** 组件，选择它们是为了对比理论，而非为了具有代表性：

- **C1**——一个一次性随机数规程（**nonce-discipline**）组件。其契约在序列上带量词；使某个特定的不安全状态不可构造（unconstructible）。量词密集。
- **C2**——一个财务成本减免组件。税务算术；金额为整数最小货币单位（`Long_Long_Integer`），无浮点，无定点类型。**gnatprove** 报告 9 项检查、0 项未证明、0 项被豁免（justified）。

5. 结果

	C1 (含量词)	C2 (整数算术)
验证条件数	18	13
无漏洞凭证数	15	13
独立校验通过 (correct)	15 (83%)	13 (100%)
平均校验时间	~0.03 秒	~0.03 秒

C1 的一条后置条件所对应的一份 626 步凭证，在 32 毫秒内被校验通过。

那三个失败其实是同一个缺陷。它们全都出现在 C1 的同一处源代码位置（一条后置条件及其两个合取项），且每份凭证都恰好包含一个未被论证的步骤。在每种情形下，那个未被论证的义务都是一次量词重写，它把一个全称量化公式与一个析取形式相关联。因此，阻碍并不是“SPARK 证明无法被认证”，而是该演算在此粒度下未予论证的单独一处重写。

理论比规模更重要。整数算术组件被完整认证；含量词的那个则没有。若这一点可以推广，那么最适合被认证的组件，恰恰是那些处理金钱、分配、边界与框架条件（framing）的组件——一个具有重大商业意义的类别。

6. 实践中的风险

6.1 版本耦合。求解器、签名集与校验器必须是同时代的。一个当前版本的校验器会拒绝较旧的签名（框架中移除了某条命令）；当前版本的签名会拒绝较旧的证明（某个步骤无法通过校验）。在有一个组合成功之前，先有三个组合失败。任何部署都必须把这三者钉在一起（pinned）发布，因为失败呈现出来的样子是证明无效，而非 版本不匹配——在安全场景中，这是一种危险的混淆。

6.2 一个看似失败的良性错误。Why3 会在其 VC 文件末尾追加一条 `get-info` 命令。cvc5 会在产出一份完整反驳之后，为此发出一个错误。若不加处理，这个末尾错误会导致校验器拒绝一份原本有效的凭证。

7. 局限与对有效性的威胁

我们把这些直白地列出，因为这份成果很容易被过度解读。

1. 凭证只覆盖 **SMT** 这一环。它确立的是这些验证条件为真（valid）。它并不确立这些条件忠实地表示了源程序：此处 `Ada`→`Why3` 的翻译与 `VC` 的生成仍是被信任的、未经认证的。对错误义务的一份有效证明，仍然毫无价值。对 Why3 核心加以形式化的工作是存在的，也是显而易见的互补方向。
2. 校验器本身未经验证。它体量小且独立，但它处于可信基之中。
3. 只有两个组件。不主张任何普遍性；理论上的对比只是有所提示，并未被确立。
4. 排除了浮点。无漏洞的 CPC 生成目前尚不能延伸到浮点（FP）算术；使用浮点的组件不在讨论范围之内。
5. 我们对相关工作的调研（§7a）是仓促进行的，并不系统。我们没有找到此前有任何报告，讲某个 SPARK 组件随附了一份经独立校验的 SMT 凭证；若有读者知道相反的情况，我们欢迎指正。

7a. 相关工作

携带证明代码（Proof-carrying code）。这一想法源自 Nacula 与 Lee (POPL 1997)：宿主通过校验一份随附凭证，来判定来自不受信任来源的代码是否可以安全运行。这条脉络经由类型化汇编语言（Typed Assembly Language, Morrisett、Walker 等，1999）、面向 Java 的认证式编译器（Colby、

Lee、Necula 等, 2000), 再到基础性 **PCC (Foundational PCC)** (Appel, 普林斯顿, 1999–2005) 而发展; 后者通过从逻辑之根基出发、不使用任何与类型相关的公理来推导证明, 从而缩小了可信基——代价是, 按该项目自身的评估, 这些证明构造起来要困难得多。后续工作把该方法延伸到自修改代码 (Cai、Shao、Vaynberg, 2007), 以及带抢占式线程与中断的底层程序 (Feng、Shao、Guo、Dong, 2009); 携带抽象代码 (Abstraction-Carrying Code, Albert、Puebla、Hermenegildo) 则处理凭证体积的问题。

关于 PCC 为何未能走入实践, 标准的说法在供给侧: 产出证明才是路障, 需要为每件制品付出专家级的努力。我们的贡献并非一个新想法, 而是关于那一约束的一项观察: 在工业代码的验证已然自动化之处, 凭证几乎是免费的, 于是那条历史性的反对意见便不再成立。

这一想法目前正被重新用于机器生成的代码——见《携带证明的代码补全》(*Proof-Carrying Code Completions*, Kamran、斯坦福等, ASEW 2024)。

认证验证器本身。VST 与 CakeML 走的是基础性路线, 把工具链实现现在一个证明助手 (proof assistant) 之内。与我们尚未闭合的那一环最接近的, 是 Cohen 与 Johnson-Freyd 的《**Core Why3 在 Coq 中的一个形式化**》(**A Formalization of Core Why3 in Coq**, POPL 2024), 该文为 Why3 的逻辑片段给出了 Coq 语义、一个构造即正确 (correct-by-construction) 的证明系统, 以及对两个 Why3 变换的可靠性 (soundness) 证明。那项工作恰好处理了我们的凭证所未触及的那一层。

SMT 证明格式与校验器。Alethe (Schurr、Fleury、Barbosa、Fontaine, EPTCS 336, 2021) 连同 Carcara 校验器 (TACAS 2023); LFSC; 以及本文所用的演算——协作证明演算 (Cooperating Proof Calculus, CAV 2026), 处于 Eunoia 框架中, 由 Ethos 校验 (IJCAR 2026)。Carcara 的作者指出, 校验器本身就处于可信基之中; 这一告诫同样适用于 Ethos, 也适用于我们。

SPARK 的实践。AdaCore 的《对一条工业级编译与静态验证工具链的聚焦式认证》(*Focused Certification of an Industrial Compilation and Static Verification Toolchain*) 记载了针对该工具链自身的资质认证 (qualification) 方法。与本文同时期, Philipp 的《证明器即裁判: 由 AI 编码代理产出的 Ada/SPARK 可验证安全软件》(**The Prover Is the Judge: Verified Security Software from AI Coding Agents in Ada/SPARK**, arXiv 2607.14340, 2026 年 7 月) 报告了由 AI 代理产出的裸机安全软件——TLS 1.3、IKEv2、X.509、后量子密码学——其中由 gnatprove 消解了 49,280 项义务。那项工作在规模上远大于我们, 且面向生产环境; 但它并未为所得制品附加可独立校验的凭证, 而这正是我们此处所要处理的缺口。两者是互补的: 验证越是自动化, 由谁来校验校验器的裁决这一问题就越发重要。

8. 结论

SPARK 验证中的求解器环节, 如今就可以被做成可独立校验的——用的是已在使用中的工具链, 校验开销可忽略不计——对整数算术组件是完全做到, 对一个含量词的组件也做到了绝大多数。剩下的问题不在求解器, 而在前端: 认证“所证明的义务正是所意图的义务”。那才是实质性的开放问题, 也正是一个“已验证”组件中, 剩余信任真正所在之处。

制品 (Artifact)

本报告随附一个自校验 (self-verifying) 的捆绑包: 源代码、证明摘要、全部凭证、相匹配的签名集, 以及一段可在读者机器上重新校验每一项义务的脚本。

参考文献

携带证明代码 (Proof-carrying code)

- [1] G. C. Necula. *Proof-Carrying Code*. In Proc. 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97), Paris, pp. 106 – 119, 1997. DOI: 10.1145/263699.263712. (2007 年获评 POPL 1997 最具影响力论文奖。)
- [2] G. C. Necula and P. Lee. *Safe Kernel Extensions Without Run-Time Checking*. In Proc. 2nd USENIX Symposium on Operating Systems Design and Implementation (OSDI '96), 1996. ——该方法首次出现于其中的前身工作。
- [3] A. W. Appel. *Foundational Proof-Carrying Code*. In Proc. 16th Annual IEEE Symposium on Logic in Computer Science (LICS '01), 2001. ——将可信基缩小至逻辑之根基，代价是证明构造上大为增加。
- [4] G. Morrisett, D. Walker, K. Crary and N. Glew. *From System F to Typed Assembly Language*. ACM Transactions on Programming Languages and Systems 21(3), 1999.

认证验证器本身

- [5] J. M. Cohen and P. Johnson-Freyd. *A Formalization of Core Why3 in Coq*. Proceedings of the ACM on Programming Languages 8(POPL), pp. 1789–1818, January 2024. DOI: 10.1145/3632902. ——为 Why3 的逻辑片段给出形式化的 Coq 语义、一个构造即正确的证明系统，以及对 Why3 两个变换的可靠性证明。恰好处理了本报告的凭证所未触及的那一层。
- [6] *A Framework for Proof-Carrying Logical Transformations*. arXiv:2107.02352.

SMT 证明格式与校验器

- [7] H.-J. Schurr, M. Fleury, H. Barbosa and P. Fontaine. *Alethe: Towards a Generic SMT Proof Format*. Proceedings of the 7th Workshop on Proof eXchange for Theorem Proving (PxTP), EPTCS 336, 2021.
- [8] B. Andreotti, H. Lachnitt and H. Barbosa. *Carcara: An Efficient Proof Checker and Elaborator for SMT Proofs in the Alethe Format*. In Proc. TACAS 2023. ——注意作者自己的告诫：校验器本身处于可信基之中；同样的告诫适用于 Ethos，也适用于本工作。
- [9] *Cooperating Proof Calculus*. CAV 2026. ——585 条证明规则，覆盖 cvc5 在其主流理论中所用的推理，以 Eunoia 框架表达。
- [10] *Ethos: An Efficient Proof Checker for the Eunoia Logical Framework*. IJCAR 2026. 实现: github.com/cvc5/ethos.
- [11] cvc5 文档，CPC 证明输出。 cvc5.github.io/docs ——声明：除涉及浮点算术者外，对 SMT-LIB 基准所产出的证明是无漏洞的。

SPARK 与可验证软件的实践

- [12] AdaCore. *Focused Certification of an Industrial Compilation and Static Verification Toolchain*. ——针对 SPARK 工具链自身的资质认证方法。
- [13] J. Philipp. *The Prover Is the Judge: Verified Security Software from AI Coding Agents in Ada/SPARK*. arXiv:2607.14340, July 2026. ——由 AI 代理产出的裸机安全软件 (TLS 1.3、IKEv2、X.509、后量子)，由 gnatprove 消解了 49,280 项义务。与本文同时期，规模远大得多，且互补：它并未为所得制品附加可独立校验的凭证。
- [14] *Proof-Carrying Code Completions*. ASEW 2024. ——面向机器生成代码而被重新审视的 PCC 想法。

文献说明。条目 [1]、[2]、[5]、[7] 与 [13] 已对照第一手来源核实。条目 [3]、[4]、[6]、[8]–[12] 与 [14] 引自第二手来源，其页码、作者列表与出处应在任何超出预印本存放（preprint deposit）的正式投稿之前，对照出版方记录加以确认。

工具与 AI 使用声明

此处所报告的实验，以及本报告的起草，均在一次借助 AI 的工作会话中完成，使用的是 Anthropic 的 Claude（Claude Code, Opus 5）。作者是神经多样性（neurodivergent）人士并远程工作；使用 AI 辅助是一种无障碍便利安排，正如另一位研究者可能会与一位誊写员或抄写助手（amanuensis）合作那样。

依据现行的发表场所政策，AI 系统不是本工作的作者，也不作为作者列名。作者对其内容承担全部责任。

关于这种辅助能影响什么、不能影响什么，有两点需要说明。§5 中的测量结果是由具名工具在钉定版本下产出的——gnatprove、cvc5 1.2.1、提交号为 45e29dd 的 Ethos——且可由任何读者从随附制品中复现，与产生这些结果的那次会话是如何进行的无关。相反，§7 中的判断——即凭证并未确立什么，以及信任究竟还剩在何处——才是读者应当审视的部分；它们被尽作者所能地表述得狭窄，正是因为 AI 辅助会让“过度声明”变得流畅而容易。

为记录各方所为：§6.1 中的版本耦合诊断，是在三次失败尝试之后由工具方提出的。而 §5 中选择报告那三个失败（而非略去它们）的决定，以及 §7 中每一项声明的范围，都出自作者本人。